

OS LAB PROGRAM

- 1) Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Process creation failed.\n");
    }
    else if (pid == 0)
    {
        printf("\n----- Child Process ----- \n");
        printf("Child Process ID : %d\n", getpid());
        printf("Parent Process ID : %d\n", getppid());
    }
    else
    {
        printf("\n----- Parent Process ----- \n");
        printf("Parent Process ID : %d\n", getpid());
        printf("Child Process ID : %d\n", pid);
    }
}
```

```
----- Parent Process -----
Parent Process ID : 16352
Child Process ID : 16353

----- Child Process -----
Child Process ID : 16353
Parent Process ID : 1

=== Code Execution Successful ===
```

- 2) Identify the system calls to copy the content of one file to another and illustrate the same using a C program.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    char sourceFile[50], destinationFile[50];
    int source, destination;
    char buffer[100];
    int bytesRead;

    printf("Enter source file name: ");
    scanf("%s", sourceFile);

    printf("Enter destination file name: ");
    scanf("%s", destinationFile);

    source = open(sourceFile, O_RDONLY);

    if (source == -1)
    {
        printf("Source file not found.\n");
    }
}
```

```
Enter source file name: input.txt
Enter destination file name: output.txt
Source file not found.

=== Code Exited With Errors ===
```

- 3) Design a CPU scheduling program with C using First Come First Served technique with the following considerations. a. All processes are activated at time 0. b. Assume that no process waits on I/O devices.

```
#include <stdio.h>

int main()
{
    int n;
    int bt[20], wt[20], tat[20];
    float avgWT = 0, avgTAT = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
    {
        printf("Enter Burst Time of P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;

    for(int i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
}
```

```
Enter number of processes: 7
Enter Burst Time of P1: 5
Enter Burst Time of P2: 4
Enter Burst Time of P3: 3
Enter Burst Time of P4: 2
Enter Burst Time of P5: 1
Enter Burst Time of P6: 8
Enter Burst Time of P7: 7

Process Burst Time Waiting Time Turnaround Time
P1 5 0 5
P2 4 5 9
P3 3 9 12
P4 2 12 14
P5 1 14 15
P6 8 15 23
P7 7 23 30

Average Waiting Time = 11.14
Average Turnaround Time = 15.43
```

- 4) Construct a scheduling program with C that selects the waiting process with the smallest execution time to execute next

```
#include <stdio.h>

int main()
{
    int n;
    int bt[20], pr[20], wt[20], tat[20], p[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
    {
        p[i] = i + 1;

        printf("Enter Burst Time of P%d: ", i + 1);
        scanf("%d", &bt[i]);

        printf("Enter Priority of P%d (Lower number = Higher Priority): ", i + 1);
        scanf("%d", &pr[i]);
    }
}
```

```
Enter number of processes: 4
Enter Burst Time of P1: 3
Enter Priority of P1 (Lower number = Higher Priority): 6
Enter Burst Time of P2: 3
Enter Priority of P2 (Lower number = Higher Priority): 4
4
4
Enter Burst Time of P3: Enter Priority of P3 (Lower number = Higher Priority): Enter
Burst Time of P4: 4
Enter Priority of P4 (Lower number = Higher Priority): 6
6

Process Priority Burst Time Waiting Time Turnaround Time
P2 4 3 0 3
P3 4 4 3 7
P1 6 3 7 10
P4 6 4 10 14

Average Waiting Time = 5.00
Average Turnaround Time = 8.50
```

- 5) Construct a scheduling program with C that selects the waiting process with the highest priority to execute next.

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[20], pr[20], wt[20], tat[20], p[20];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("\nEnter Burst Time and Priority for Process %d: ", p[i]);
        scanf("%d %d", &bt[i], &pr[i]);
    }

    // Sort based on priority (lower number = higher priority)
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(pr[j] < pr[i])
            {
                int temp_bt = bt[i];
                int temp_pr = pr[i];
                bt[i] = bt[j];
                pr[i] = pr[j];
                bt[j] = temp_bt;
                pr[j] = temp_pr;
            }
        }
    }
}
```

```
Enter the number of processes: 3
Enter Burst Time and Priority for Process 1: 5 6
Enter Burst Time and Priority for Process 2: 4 7
Enter Burst Time and Priority for Process 3: 3 2

Process BT Priority WT TAT
P3 3 2 0 3
P1 5 6 3 8
P2 4 7 8 12

Average Waiting Time = 3.67
Average Turnaround Time = 7.67

=== Code Execution Successful ===
```

6) Construct a C program to implement preemptive priority scheduling algorithm

```

int main() {
    int n, bt[10], at[10], pr[10], rt[10], ct[10];
    int completed = 0, time = 0, i, min, p;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("P%d Arrival Time, Burst Time, Priority: ", i + 1);
        scanf("%d%d%d", &at[i], &bt[i], &pr[i]);
        rt[i] = bt[i];
    }
    while(completed < n) {
        min = 9999;
        p = -1;
        for(i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && pr[i] < min) {
                min = pr[i];
                p = i;
            }
        }
        if(p == -1) {
            time++;
            continue;
        }
        rt[p]--;
        time++;

        if(rt[p] == 0) {
            ct[p] = time;
            completed++;
        }
    }
}

```

P1 Arrival Time, Burst Time, Priority: 1 2
P2 Arrival Time, Burst Time, Priority: 4 5
P3 Arrival Time, Burst Time, Priority: 7 8

Process Completion Time
P1 3
P2 9
P3 17

7) Construct a C program to implement a non-preemptive SJF algorithm.

```

int main() {
    int n, bt[10], wt[10], tat[10], i, j, temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort burst times
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    wt[0] = 0;
    for(i = 1; i < n; i++)
        wt[i] = wt[i - 1] + bt[i - 1];

    for(i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];
}

```

Enter Burst Time for P1: 4
Enter Burst Time for P2: 5
Enter Burst Time for P3: 6

Process	BT	WT	TAT
P1	4	0	4
P2	5	4	9
P3	6	9	15

8) Construct a C program to simulate Round Robin scheduling algorithm with C.

```

1  int main() {
2      int n, bt[10], rt[10], tq, i, time = 0, done;
3
4      printf("Enter number of processes: ");
5      scanf("%d", &n);
6
7      for(i = 0; i < n; i++) {
8          printf("Enter Burst Time for P%d: ", i + 1);
9          scanf("%d", &bt[i]);
10         rt[i] = bt[i];
11     }
12
13     printf("Enter Time Quantum: ");
14     scanf("%d", &tq);
15
16     printf("\nProcess\tCompletion Time\n");
17
18     do {
19         done = 1;
20         for(i = 0; i < n; i++) {
21             if(rt[i] > 0) {
22                 done = 0;
23
24                 if(rt[i] > tq) {
25                     time += tq;
26                     rt[i] -= tq;
27                 } else {
28                     time += rt[i];
29                     rt[i] = 0;
30                 }
31             }
32         }
33     } while(!done);
34 }

```

Enter Burst Time for P1: 8
Enter Burst Time for P2: 4
Enter Time Quantum: 2

Process Completion Time
P2 8
P1 12

9) Illustrate the concept of inter-process communication using shared memory with a C program.

```

1  #include <stdio.h>
2  #include <sys/ipc.h>
3  #include <sys/shm.h>
4  #include <string.h>
5
6  int main() {
7      key_t key = 1234;
8      int shmid;
9      char *data;
10
11     // Create shared memory
12     shmid = shmget(key, 1024, 0666 | IPC_CREAT);
13
14     // Attach shared memory
15     data = (char *)shmat(shmid, NULL, 0);
16
17     // Write data
18     strcpy(data, "Hello from Shared Memory!");
19     printf("Data written: %s\n", data);
20
21     // Detach shared memory
22     shmdt(data);
23
24     return 0;
25 }

```

Data written: Hello from Shared Memory

10) Illustrate the concept of inter-process communication using message queue with a C program.

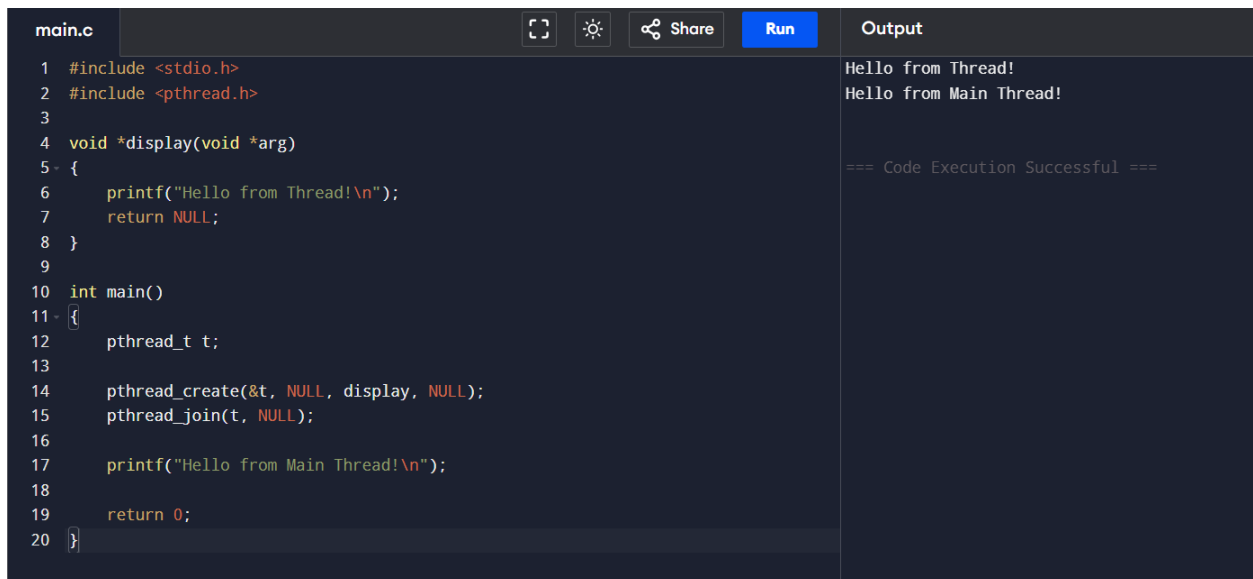
A screenshot of a code editor showing a C program that demonstrates inter-process communication using a message queue. The code includes headers for stdio, sys/ipc, sys/msg, and string. It defines a struct msgbuf with fields for message type and text. The main function creates a message queue, sends a message "Hello from Memory!", and then receives it, printing the received message. The console output shows "Received Message: Hello from Memory".

```
1 #include <stdio.h>
2 #include <sys/ipc.h>
3 #include <sys/msg.h>
4 #include <string.h>
5
6 struct msgbuf {
7     long mtype;
8     char mtext[100];
9 };
10
11 int main() {
12     struct msgbuf msg;
13     int msgid;
14
15     // Create message queue
16     msgid = msgget(1234, 0666 | IPC_CREAT);
17
18     // Send message
19     msg.mtype = 1;
20     strcpy(msg.mtext, "Hello from Memory!");
21     msgsnd(msgid, &msg, sizeof(msg.mtext), 0);
22
23     // Receive message
24     msgrcv(msgid, &msg, sizeof(msg.mtext), 1, 0);
25
26     printf("Received Message: %s\n", msg.mtext);
27
28     return 0;
29 }
```

Console

Received Message: Hello from Memory

11) Illustrate the concept of multithreading using a C program.

A screenshot of a code editor showing a C program that demonstrates multithreading using pthreads. The code includes headers for stdio and pthread. It defines a function display that prints "Hello from Thread!". The main function creates a thread, joins it, and then prints "Hello from Main Thread!". The output shows "Hello from Thread!", "Hello from Main Thread!", and "=== Code Execution Successful ===".

```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3
4 void *display(void *arg)
5 {
6     printf("Hello from Thread!\n");
7     return NULL;
8 }
9
10 int main()
11 {
12     pthread_t t;
13
14     pthread_create(&t, NULL, display, NULL);
15     pthread_join(t, NULL);
16
17     printf("Hello from Main Thread!\n");
18
19     return 0;
20 }
```

Output

Hello from Thread!
Hello from Main Thread!

=== Code Execution Successful ===

12) Design a C program to simulate the concept of Dining-Philosophers problem

```
main.c  [Icons] [Run] [Output]
1
2 #include <stdio.h>
3 #include <pthread.h>
4 pthread_mutex_t chopstick = PTHREAD_MUTEX_INITIALIZER;
5 void *philosopher(void *arg)
6 {
7     int id = *(int *)arg;
8     pthread_mutex_lock(&chopstick);
9     printf("Philosopher %d is Eating\n", id);
10    pthread_mutex_unlock(&chopstick);
11    return NULL;
12 }
13 int main()
14 {
15     pthread_t p[5];
16     int id[5] = {1, 2, 3, 4, 5};
17     for (int i = 0; i < 5; i++)
18         pthread_create(&p[i], NULL, philosopher, &id[i]);
19     for (int i = 0; i < 5; i++)
20         pthread_join(p[i], NULL);
21     return 0;
22 }
```

Philosopher 1 is Eating
Philosopher 2 is Eating
Philosopher 4 is Eating
Philosopher 5 is Eating
Philosopher 3 is Eating

=== Code Execution Successful ===

13) Construct a C program for implementation of the various memory allocation strategies.

```
main.c  [Icons] [Run] [Output]
1 #include <stdio.h>
2
3 int main()
4 {
5     int block[5] = {100, 500, 200, 300, 600};
6     int process[4] = {212, 417, 112, 426};
7
8     for (int i = 0; i < 4; i++)
9     {
10         for (int j = 0; j < 5; j++)
11         {
12             if (block[j] >= process[i])
13             {
14                 printf("Process %d allocated to Block %d\n", i + 1, j + 1);
15                 block[j] -= process[i];
16                 break;
17             }
18         }
19     }
20
21     return 0;
22 }
```

Process 1 allocated to Block 2
Process 2 allocated to Block 5
Process 3 allocated to Block 2

=== Code Execution Successful ===

14) Construct a C program to organise the file using a single level directory.

main.c	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 int n; 6 char file[10][20]; 7 8 printf("Enter number of files: "); 9 scanf("%d", &n); 10 11 printf("Enter file names:\n"); 12 for (int i = 0; i < n; i++) 13 scanf("%s", file[i]); 14 15 printf("\nFiles in Single Level Directory:\n"); 16 for (int i = 0; i < n; i++) 17 printf("%s\n", file[i]); 18 19 return 0; 20 }</pre>	<pre>Enter number of files: 2 Enter file names: file 1 Files in Single Level Directory: file 1 === Code Execution Successful ===</pre>

15) Design a C program to organise the file using a two level directory structure.

main.c	Output
<pre>5 int users, files; 6 7 printf("Enter number of users: "); 8 scanf("%d", &users); 9 10 for (int i = 0; i < users; i++) 11 { 12 printf("\nEnter number of files for User %d: ", i + 1); 13 scanf("%d", &files); 14 15 printf("Files of User %d:\n", i + 1); 16 17 for (int j = 0; j < files; j++) 18 { 19 char name[20]; 20 scanf("%s", name); 21 printf("%s\n", name); 22 } 23 } 24 25 return 0; 26 }</pre>	<pre>Enter number of users: 2 Enter number of files for User 1: file 1 Files of User 1: Enter number of files for User 2: Files of User 2: === Code Execution Successful ===</pre>

16) Develop a C program for implementing random access file for processing the employee details.

```
main.c [ ] [ ] [ ] Share Run Output
1 #include <stdio.h>
2 struct employee
3 {
4     int id;
5     char name[20];
6     float salary;
7 };
8 int main()
9 {
10     FILE *fp;
11     struct employee e;
12     int n, pos;
13     fp = fopen("emp.dat", "w+");
14     printf("Enter number of employees: ");
15     scanf("%d", &n);
16     for(int i = 0; i < n; i++)
17     {
18         printf("Enter ID, Name, Salary: ");
19         scanf("%d %s %f", &e.id, e.name, &e.salary);
20         fwrite(&e, sizeof(e), 1, fp);
21     }
22 }
```

Enter number of employees: 2
Enter ID, Name, Salary:
101 arun 10000
Segmentation fault

=== Code Exited With Errors ===

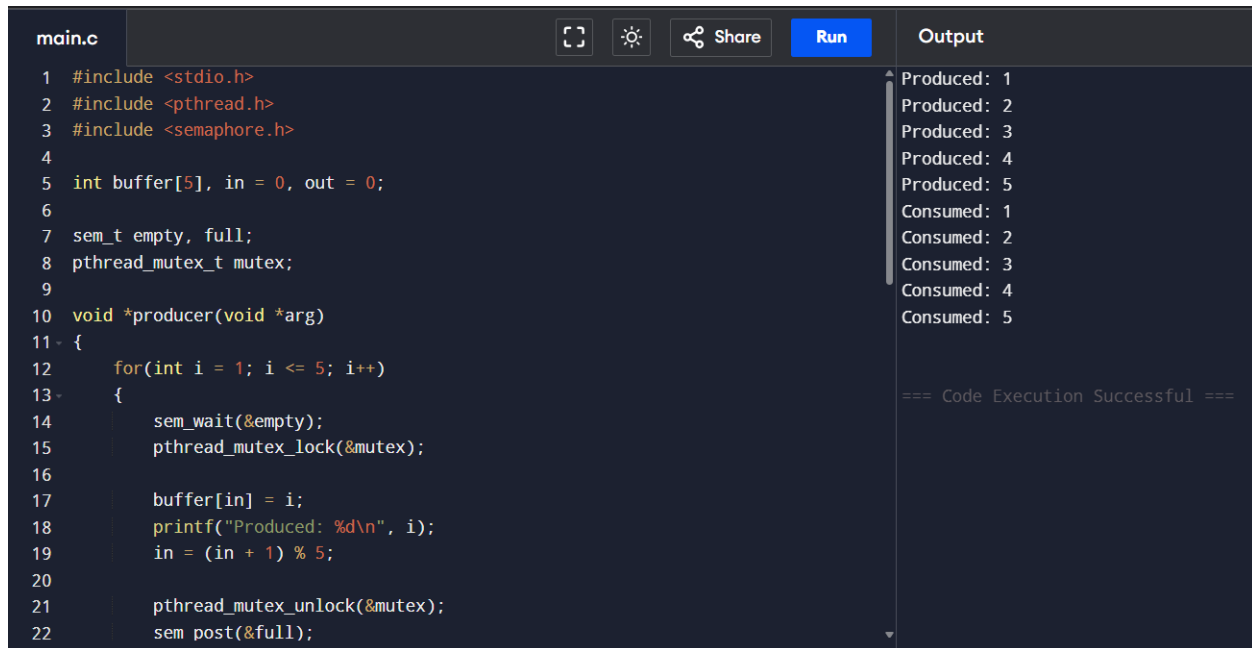
17) Illustrate the deadlock avoidance concept by simulating Banker's algorithm with C.

```
main.c [ ] [ ] [ ] Share Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     int n = 5, m = 3;
6     int alloc[5][3] = {{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};
7     int max[5][3] = {{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
8     int avail[3] = {3,3,2};
9     int need[5][3], finish[5]={0}, safe[5];
10
11     for(int i=0;i<n;i++)
12         for(int j=0;j<m;j++)
13             need[i][j]=max[i][j]-alloc[i][j];
14
15     int count=0;
16     while(count<n)
17     {
18         int found=0;
19
20         for(int i=0;i<n;i++)
21         {
22             if(!finish[i])
```

System is in safe state
Safe sequence: P1 P3 P4 P0 P2

=== Code Execution Successful ===

18) Construct a C program to simulate producer-consumer problem using semaphores.



The screenshot shows a C program in a code editor. The code defines a buffer of size 5, an 'in' pointer, and an 'out' pointer. It uses two semaphores, 'empty' and 'full', and a mutex to manage access to the buffer. The 'producer' function produces 5 items, and the 'consumer' function consumes 5 items. The output shows the sequence of production and consumption.

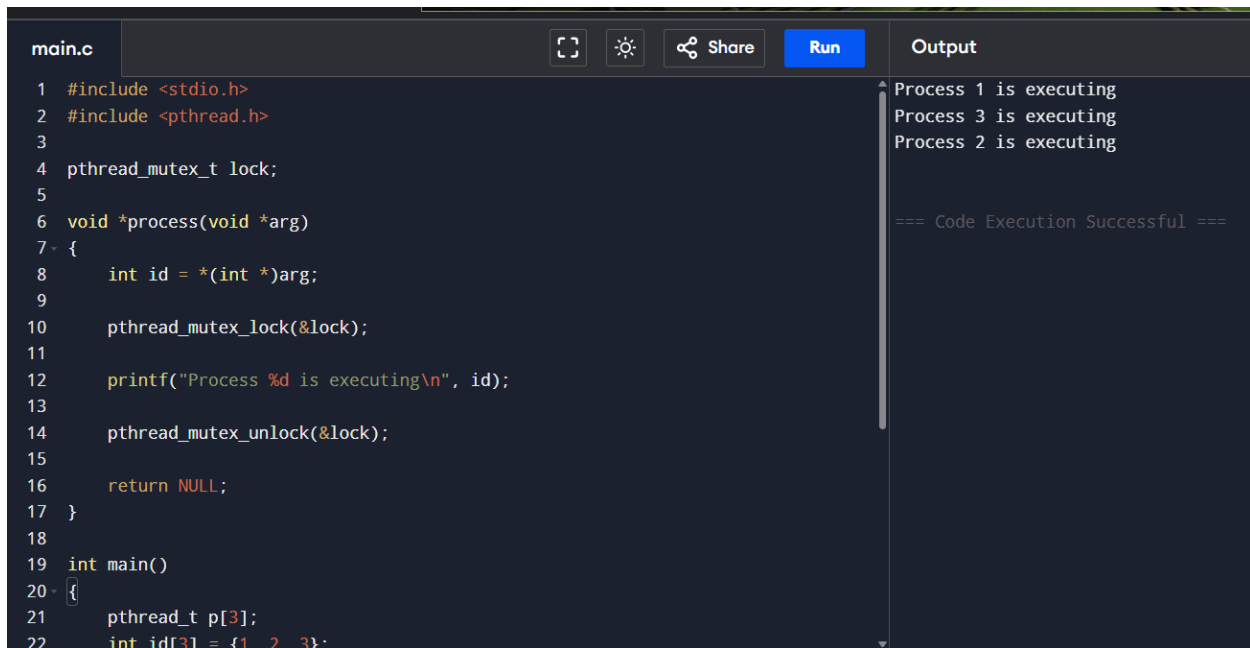
```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4
5 int buffer[5], in = 0, out = 0;
6
7 sem_t empty, full;
8 pthread_mutex_t mutex;
9
10 void *producer(void *arg)
11 {
12     for(int i = 1; i <= 5; i++)
13     {
14         sem_wait(&empty);
15         pthread_mutex_lock(&mutex);
16
17         buffer[in] = i;
18         printf("Produced: %d\n", i);
19         in = (in + 1) % 5;
20
21         pthread_mutex_unlock(&mutex);
22         sem_post(&full);
23     }
24 }
```

Output:

```
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5

=== Code Execution Successful ===
```

19) Design a C program to implement process synchronization using mutex locks.



The screenshot shows a C program in a code editor. It defines a mutex lock and a 'process' function that prints the process ID. The 'main' function creates three threads and starts them. The output shows the execution order of the three processes.

```
main.c
1 #include <stdio.h>
2 #include <pthread.h>
3
4 pthread_mutex_t lock;
5
6 void *process(void *arg)
7 {
8     int id = *(int *)arg;
9
10    pthread_mutex_lock(&lock);
11
12    printf("Process %d is executing\n", id);
13
14    pthread_mutex_unlock(&lock);
15
16    return NULL;
17 }
18
19 int main()
20 {
21     pthread_t p[3];
22     int id[3] = {1, 2, 3};
23
24     for(int i = 0; i < 3; i++)
25     {
26         pthread_create(&p[i], NULL, process, &id[i]);
27     }
28
29     for(int i = 0; i < 3; i++)
30     {
31         pthread_join(p[i], NULL);
32     }
33 }
```

Output:

```
Process 1 is executing
Process 3 is executing
Process 2 is executing

=== Code Execution Successful ===
```

20) Construct a C program to simulate Reader-Writer problem using Semaphores.

main.c	Output
<pre> 1 #include <stdio.h> 2 #include <pthread.h> 3 #include <semaphore.h> 4 5 sem_t mutex, writeblock; 6 int data = 0, readcount = 0; 7 8 void *reader(void *arg) 9 { 10 int id = *(int *)arg; 11 12 sem_wait(&mutex); 13 readcount++; 14 if(readcount == 1) 15 sem_wait(&writeblock); 16 sem_post(&mutex); 17 18 printf("Reader %d reads data = %d\n", id, data); 19 20 sem_wait(&mutex); 21 readcount--; 22 if(readcount == 0) </pre>	<pre> Reader 1 reads data = 0 Reader 2 reads data = 0 Reader 3 reads data = 0 Writer 1 writes data = 1 Writer 2 writes data = 2 === Code Execution Successful === </pre>

21) Develop a C program to implement the worst fit algorithm of memory management.

main.c	Output
<pre> 1 #include <stdio.h> 2 3 int main() 4 { 5 int block[5] = {100, 500, 200, 300, 600}; 6 int process[4] = {212, 417, 112, 426}; 7 8 for(int i = 0; i < 4; i++) 9 { 10 int worst = -1; 11 12 for(int j = 0; j < 5; j++) 13 { 14 if(block[j] >= process[i]) 15 { 16 if(worst == -1 block[j] > block[worst]) 17 worst = j; 18 } 19 } </pre>	<pre> Process 1 allocated to Block Process 2 allocated to Block Process 3 allocated to Block Process 4 not allocated === Code Execution Successful === </pre>

22) Construct a C program to implement the best fit algorithm of memory management.

```
main.c [ ] [ ] [ ] Share Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     int block[5] = {100, 500, 200, 300, 600};
6     int process[4] = {212, 417, 112, 426};
7
8     for(int i = 0; i < 4; i++)
9     {
10         int best = -1;
11
12         for(int j = 0; j < 5; j++)
13         {
14             if(block[j] >= process[i])
15             {
16                 if(best == -1 || block[j] < block[best])
17                     best = j;
18             }
19         }
20
21         if(best != -1)
22         {
23             printf("Process %d allocated to Block %d\n", i+1, best+1);
24             block[best] -= process[i];
25         }
26     }
27
28     return 0;
29 }
```

Process 1 allocated to Block 4
Process 2 allocated to Block 2
Process 3 allocated to Block 3
Process 4 allocated to Block 5

=== Code Execution Successful ===

23) Construct a C program to implement the first fit algorithm of memory management.

```
main.c [ ] [ ] [ ] Share Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     int block[5] = {100, 500, 200, 300, 600};
6     int process[4] = {212, 417, 112, 426};
7
8     for(int i = 0; i < 4; i++)
9     {
10         for(int j = 0; j < 5; j++)
11         {
12             if(block[j] >= process[i])
13             {
14                 printf("Process %d allocated to Block %d\n", i+1, j+1);
15                 block[j] -= process[i];
16                 break;
17             }
18         }
19     }
20
21     return 0;
22 }
```

Process 1 allocated to Block 2
Process 2 allocated to Block 5
Process 3 allocated to Block 2

=== Code Execution Successful ===

24) Design a C program to demonstrate UNIX system calls for file management.

```
main.c  [Icons]  Run  Output  Clear
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4 #include <sys/stat.h>
5 #include <dirent.h>
6
7 int main()
8 {
9     int fd;
10    struct stat st;
11    DIR *dir;
12    struct dirent *entry;
13
14    // fcntl system call
15    fd = open("file.txt", O_CREAT | O_RDWR, 0777);
16    fcntl(fd, F_SETFL, O_APPEND);
17
18    write(fd, "UNIX System Call", 16);
19
20    // lseek system call
21    lseek(fd, 0, SEEK_SET);
22    printf("File Content: %s\n", buffer);
23
24    close(fd);
25
26    return 0;
27 }
```

A module you have imported isn't available at the moment. It will be available soon.

26) Construct a C program to implement the file management operations.

```
main.c  [Icons]  Run  Output
1 #include <stdio.h>
2
3 int main() {
4     FILE *fp;
5     char ch;
6
7     // Write to file
8     fp = fopen("test.txt", "w");
9     fprintf(fp, "Hello File!");
10    fclose(fp);
11
12    // Read from file
13    fp = fopen("test.txt", "r");
14    while ((ch = fgetc(fp)) != EOF)
15        printf("%c", ch);
16    fclose(fp);
17
18    // Append to file
19    fp = fopen("test.txt", "a");
20    fprintf(fp, "\nWelcome");
21    fclose(fp);
22 }
```

Segmentation fault

=== Code Exited With Errors ===

27) Develop a C program for simulating the function of ls UNIX Command.

main.c	Output
<pre>1 #include <stdio.h> 2 #include <dirent.h> 3 4 int main() { 5 DIR *d; 6 struct dirent *dir; 7 8 d = opendir("."); 9 if (d == NULL) { 10 printf("Cannot open directory"); 11 return 1; 12 } 13 14 while ((dir = readdir(d)) != NULL) 15 printf("%s\n", dir->d_name); 16 17 closedir(d); 18 return 0; 19 }</pre>	<pre>. .. .bash_logout .bashrc .profile === Code Execution Successful ===</pre>

28) Write a C program for simulation of GREP UNIX command

main.c	Output
<pre>1 #include <stdio.h> 2 #include <string.h> 3 4 int main() { 5 FILE *fp; 6 char line[100], word[20]; 7 8 printf("Enter word: "); 9 scanf("%s", word); 10 11 fp = fopen("test.txt", "r"); 12 13 while (fgets(line, sizeof(line), fp)) 14 if (strstr(line, word)) 15 printf("%s", line); 16 17 fclose(fp); 18 return 0; 19 }</pre>	<pre>Enter word: hello Segmentation fault === Code Exited With Errors ===</pre>

29) Write a C program to simulate the solution of Classical Process Synchronization Problem

main.c	Output
<pre>1 #include <stdio.h> 2 3 int mutex = 1, full = 0, empty = 3; 4 5 void wait(int *s) { (*s)--; } 6 void signal(int *s) { (*s)++; } 7 8 int main() { 9 wait(&empty); 10 wait(&mutex); 11 printf("Producer produced an item\n"); 12 signal(&mutex); 13 signal(&full); 14 15 wait(&full); 16 wait(&mutex); 17 printf("Consumer consumed an item\n"); 18 signal(&mutex); 19 signal(&empty); 20 21 return 0; 22 }</pre>	<pre>Producer produced an item Consumer consumed an item === Code Execution Successful</pre>

30) Write C programs to demonstrate the following thread related concepts.

(i) create

main.c	Output
<pre>1 #include <stdio.h> 2 #include <pthread.h> 3 4 void *fun(void *arg) { 5 printf("Thread Created\n"); 6 return NULL; 7 } 8 9 int main() { 10 pthread_t t; 11 pthread_create(&t, NULL, fun, NULL); 12 pthread_join(t, NULL); 13 return 0; 14 }</pre>	<pre>Thread Created === Code Execution Successful</pre>

(ii) join

main.c	Output
<pre>1 #include <stdio.h> 2 #include <pthread.h> 3 4 void *fun(void *arg) { 5 printf("Thread Running\n"); 6 return NULL; 7 } 8 9 int main() { 10 pthread_t t; 11 pthread_create(&t, NULL, fun, NULL); 12 pthread_join(t, NULL); 13 printf("Thread Joined\n"); 14 return 0; 15 }</pre>	<pre>Thread Running Thread Joined === Code Execution Successful ===</pre>

(iii) equal

main.c	Output
<pre>1 #include <stdio.h> 2 #include <pthread.h> 3 4 void *fun(void *arg) { 5 printf("Thread Running\n"); 6 return NULL; 7 } 8 9 int main() { 10 pthread_t t; 11 pthread_create(&t, NULL, fun, NULL); 12 pthread_join(t, NULL); 13 printf("Thread Joined\n"); 14 return 0; 15 }</pre>	<pre>Thread Running Thread Joined === Code Execution Successful ===</pre>

(iv) exit

main.c	Output
<pre>1 #include <stdio.h> 2 #include <pthread.h> 3 4 void *fun(void *arg) { 5 printf("Thread Exit\n"); 6 pthread_exit(NULL); 7 } 8 9 int main() { 10 pthread_t t; 11 pthread_create(&t, NULL, fun, NULL); 12 pthread_join(t, NULL); 13 return 0; 14 }</pre>	<pre>Thread Exit === Code Execution Successful ===</pre>

31) Construct a C program to simulate the First in First Out paging technique of memory management.

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int pages[] = {1, 2, 3, 2, 4, 1, 5}; 5 int frame[3] = {-1, -1, -1}; 6 int i, j, k = 0, fault = 0, found; 7 for (i = 0; i < 7; i++) { 8 found = 0; 9 for (j = 0; j < 3; j++) 10 if (frame[j] == pages[i]) 11 found = 1; 12 if (!found) { 13 frame[k] = pages[i]; 14 k = (k + 1) % 3; 15 fault++; 16 } 17 } 18 printf("Page Faults = %d", fault); 19 return 0; 20 }</pre>		Page Faults = 6 === Code Execution Successful ===

32) Construct a C program to simulate the Least Recently Used paging technique of memory management.

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int pages[] = {1, 2, 3, 2, 4, 1, 5}; 5 int frame[3] = {-1, -1, -1}, time[3] = {0}; 6 int i, j, pos, min, fault = 0, found; 7 8 for (i = 0; i < 7; i++) { 9 found = 0; 10 for (j = 0; j < 3; j++) 11 if (frame[j] == pages[i]) { 12 found = 1; 13 time[j] = i; 14 } 15 16 if (!found) { 17 min = time[0]; 18 pos = 0; 19 for (j = 1; j < 3; j++) 20 if (time[j] < min) { 21 min = time[j]; 22 pos = j; 23 } 24 frame[pos] = pages[i]; 25 time[pos] = i; 26 fault++; 27 } 28 } 29 printf("Page Faults = %d", fault); 30 return 0; 31 }</pre>		Page Faults = 6 === Code Execution Successful ===

33) Construct a C program to simulate the optimal paging technique of memory management

main.c	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int pages[] = {1, 2, 3, 2, 4, 1, 5}; 5 int frame[3] = {-1, -1, -1}, time[3] = {0}; 6 int i, j, pos, min, fault = 0, found; 7 8 for (i = 0; i < 7; i++) { 9 found = 0; 10 for (j = 0; j < 3; j++) 11 if (frame[j] == pages[i]) { 12 found = 1; 13 time[j] = i; 14 } 15 16 if (!found) { 17 min = time[0]; 18 pos = 0; 19 for (j = 1; j < 3; j++) 20 if (time[j] < min) { 21 min = time[j]; 22 pos = j; 23 } 24 frame[pos] = pages[i]; 25 fault++; 26 } 27 } 28 printf("Page Faults = %d\n", fault); 29 return 0; 30 }</pre>	<pre>Page Faults = 6 === Code Execution Successful ===</pre>

34) Consider a file system where the records of the file are stored one after another both physically and logically. A record of the file can only be accessed by reading all the previous records. Design a C program to simulate the file allocation strategy.

main.c	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int n, i; 5 6 printf("Enter number of records: "); 7 scanf("%d", &n); 8 9 printf("Sequential File Allocation:\n"); 10 for (i = 0; i < n; i++) 11 printf("Record %d -> Block %d\n", i + 1, i + 1); 12 13 return 0; 14 }</pre>	<pre>Enter number of records: 4 Sequential File Allocation: Record 1 -> Block 1 Record 2 -> Block 2 Record 3 -> Block 3 Record 4 -> Block 4 === Code Execution Successful ===</pre>

35) Consider a file system that brings all the file pointers together into an index block. The i th entry in the index block points to the i th block of the file. Design a C program to simulate the file allocation strategy.

main.c	Output
<pre> 1 #include <stdio.h> 2 3 int main() { 4 int n, i, index; 5 6 printf("Enter Index Block: "); 7 scanf("%d", &index); 8 9 printf("Enter Number of Blocks: "); 10 scanf("%d", &n); 11 12 printf("Enter Block Numbers:\n"); 13 for (i = 1; i <= n; i++) { 14 int block; 15 scanf("%d", &block); 16 printf("Index[%d] -> Block %d\n", i, block); 17 } 18 19 return 0; 20 }</pre>	<pre> Enter Index Block: 2 Enter Number of Blocks: 2 Enter Block Numbers: 1 Index[1] -> Block 1 20 Index[2] -> Block 20 === Code Execution Successful</pre>

36) With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file. Each block contains a pointer to the next block. Design a C program to simulate the file allocation strategy.

main.c	Output
<pre> 1 #include <stdio.h> 2 3 int main() { 4 int n, i, block; 5 6 printf("Enter number of blocks: "); 7 scanf("%d", &n); 8 9 printf("Enter block numbers:\n"); 10 for (i = 0; i < n; i++) { 11 scanf("%d", &block); 12 if (i == n - 1) 13 printf("%d -> NULL\n", block); 14 else 15 printf("%d -> ", block); 16 } 17 18 return 0; 19 }</pre>	<pre> Enter number of blocks: 3 Enter block numbers: 4 4 -> 2 2 -> 1 1 -> NULL === Code Execution Successful ===</pre>

37) Construct a C program to simulate the First Come First Served disk scheduling algorithm.

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 4 int main() { 5 int req[5] = {98, 183, 37, 122, 14}; 6 int head = 53, i, seek = 0; 7 8 for (i = 0; i < 5; i++) { 9 seek = seek + abs(req[i] - head); 10 head = req[i]; 11 } 12 13 printf("Total Seek Time = %d\n", seek); 14 15 return 0; 16 }</pre>	Total Seek Time = 469 === Code Execution Successful ===

38) Design a C program to simulate SCAN disk scheduling algorithm.

main.c	Output
<pre>1 #include<stdio.h> 2 #include<stdlib.h> 3 4 int main() { 5 int a[20], n, h, i, seek = 0; 6 printf("Enter n & head: "); 7 scanf("%d%d", &n, &h); 8 9 printf("Enter requests: "); 10 for(i = 0; i < n; i++) scanf("%d", &a[i]); 11 12 for(i = 0; i < n; i++) { 13 seek += abs(h - a[i]); 14 h = a[i]; 15 } 16 17 printf("Seek Time = %d", seek); 18 return 0; 19 }</pre>	Enter n & head: 4 20 56 58 45 Enter requests: 2 Seek Time = 94 === Code Execution Successful ===

39) Develop a C program to simulate C-SCAN disk scheduling algorithm.

main.c	Output
<pre>1 #include<stdio.h> 2 #include<stdlib.h> 3 4 int main() { 5 int a[20], n, h, i, seek = 0; 6 printf("Enter n & head: "); 7 scanf("%d%d", &n, &h); 8 9 printf("Enter requests: "); 10 for(i = 0; i < n; i++) { 11 scanf("%d", &a[i]); 12 seek += abs(h - a[i]); 13 h = a[i]; 14 } 15 16 seek += 199; // Circular jump 17 printf("Seek Time = %d", seek); 18 19 return 0; 20 }</pre>	<pre>Enter n & head: 5 50 Enter requests: 25 36 45 78 96 Seek Time = 295 === Code Execution Successful ===</pre>

40) Illustrate the various File Access Permission and different types of users in Linux.

main.c	Output
<pre>1 #include <stdio.h> 2 #include <sys/stat.h> 3 4 int main() { 5 struct stat f; 6 stat("test.txt", &f); 7 8 printf("Owner Read: %d\n", (f.st_mode & S_IRUSR)!=1); 9 printf("Owner Write: %d\n", (f.st_mode & S_IWUSR)!=1); 10 printf("Owner Execute: %d\n", (f.st_mode & S_IXUSR)!=0); 11 12 return 0; 13 }</pre>	<pre>Owner Read: 1 Owner Write: 1 Owner Execute: 0 === Code Execution Successful ===</pre>